

Prefix Installation and Setup Guide

Installation:

1. Download the Prefix installer from: [<http://support.stackify.com/hc/en-us/articles/207800663-Prefix-Download-Release-Notes>]
2. Run the setup application.
3. Click the Prefix icon which appears in the system tray.
4. Click on 'Enable .NET Profiler'.
5. Start Microsoft Visual Studio, or restart it if it is already running
6. Run CW Master Teacher from Visual Studio

To open the Profiler Page (if needed) and begin viewing data:

7. Click on the Prefix icon in the system tray.
8. Click on 'Open in Browser' to open the Prefix Profiler page (localhost:2012 by default).

A Few Notes:

When the setup application has completed, the prefix profiler should appear in the system tray, and the setup application will open a couple of websites.

Prefix Profiler:

The prefix profiler allows you to:

1. open the Profiler webpage
2. enable or disable the profiler
3. open the documentation for enabling Java profiling

For the purposes of CW Master Teacher, we will be focusing on the first two of these abilities. The Profiler page is where the data Prefix gathers will be displayed while the profiler is running.

In order to use Prefix to assess CW Master Teacher, it needs to be enabled. This involves three steps:

1. Click on the Prefix icon in the system tray and then click 'Enable .NET profiler'.
2. If Visual Studio is open, restart it. If not, start it.
3. Begin running CW Master Teacher

Once the profiler is enabled, Prefix will gather data from CW Master Teacher as it runs and display that information in the profiler page.

Profiler webpage:

This page will have an address that looks like "localhost:2012" by default. The port number Prefix uses can be changed by clicking the 'settings' button which appears in an overlay when the Prefix Profiler is disabled or waiting for activity. The profiler web page is where the information that Prefix gathers is displayed. This is where most of the time spent on detecting and diagnosing performance issues will be spent. The Prefix Usage Guide will go over the different uses of this page in more detail.

Prefix thank you page:

This page has a few different useful links pointing a new user towards the next steps. These links include a tutorial video showing some of the ways Prefix can be used and some of the information it provides, and a link to the Prefix documentation- such as it is- which has some information on configuring Prefix for different setups and some troubleshooting guides.

What Is Stackify Prefix?

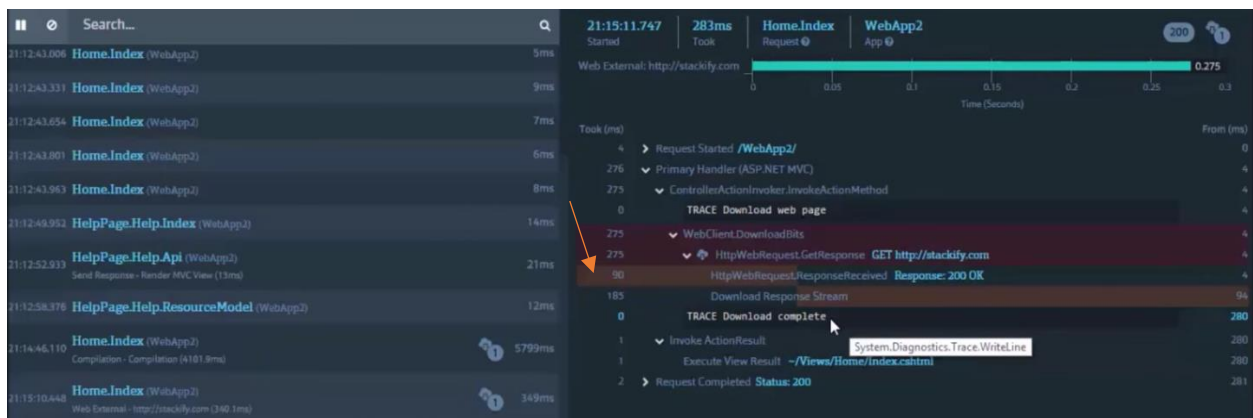
Prefix will help troubleshoot what your code is doing by inspecting key methods, database queries, web service calls, and logging statements.

Key Features:

- Find slow SQL queries, web service calls, etc.
- View logging, Debug, & Trace statements
- Runs only on a developer's workstation. No server installs.
- Supports 30+ common .NET libraries ([view list](#))

Examples

1. As in this example, requesting the webpage we can see that the web client took 275 milliseconds to complete the request (90 milliseconds to get the initial response and another 185 milliseconds to download the page).



2. There is a Prefix link what is used to demo the tool (<http://demo.prefix.io/>). Click on any of the web requests on the left side and you will get on the profile trace side all sort of information about the request such as, the type of the request, the URL, the errors (including the caught errors), debug and logging statements, caching, databases, web service calls, and so on. Because prefix is built on top of .net profiler, we can catch all thrown exceptions even without logging at all. So, click on any request from the left side and check the thrown exceptions on the right side. You will also be able to all the database queries (picture below). For example, we can copy that SQL to any SQL management tool to tune this some

Query tuning and update the code. Prefix allows us to uncover hidden amount of time spend on different sort of computations, for example, we can see that it took 23 milliseconds to run this query but 20 of those is to download the query and so, something like 3 milliseconds was to run that query (23 milliseconds of total time to iterate through all the records).

The screenshot shows the Prefix tool interface. On the left, a list of requests is displayed with their respective times and icons. The selected request is 'people.GetTopPeople' (HelloStackify.Web) with a total time of 52ms. On the right, the detailed view of this request is shown, including the SQL query and the execution steps. The 'Download Query Result' step is highlighted, showing it took 20ms.

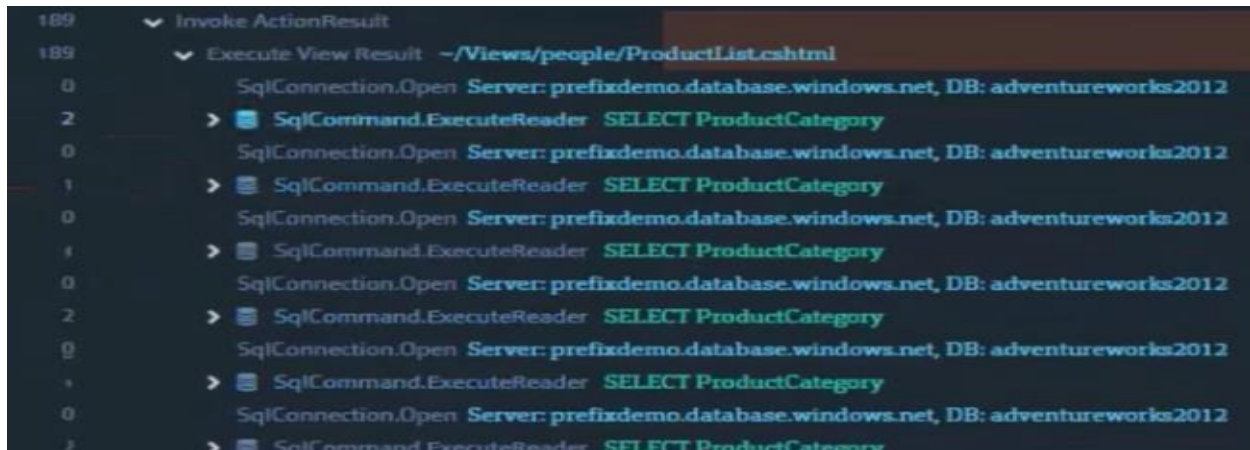
3. In this request, those icons are (Database calls, Web service calls, cache calls, and Errors). When pressing Errors, Prefix will show you all exceptions (which is very computationally expensive specially in large projects) even if you are not logging them anywhere.

The screenshot shows the Prefix tool interface with a list of requests. The selected request is 'stackify.devices' (HelloStackify.Web) with a total time of 261ms. An arrow points to the error icon in the request list.

4. Here is a good example of a common performance problem.

The screenshot shows the Prefix tool interface with a list of requests. The selected request is 'People.ProductList' (HelloStackify.Web) with a total time of 361ms. On the right, the detailed view of this request is shown, including a horizontal bar chart of execution steps. The 'DB Query: SELECT Product' step is highlighted, showing it took 0.161s. The 'Send Response: Render MVC View' step is also highlighted, showing it took 0.135s.

People.QueueSomeJson took 361 milliseconds and had 38 database calls. We can see that when it rendered this to the MVC view (to give us the product list) there is all those sort of database queries that are happening (picture below), those extra queries are because of lazy loading when using the entity framework (Lazy loading and serialization don't mix well, and if you aren't careful you can end up querying for your entire database just because lazy loading is enabled.)



-AS we can see here in this summarizing section in Prefix, the main query took 0.16 milliseconds while other queries took 0.054 milliseconds. From above we can see that rendering the view took 189 milliseconds and 0.135 of that wasn't even a database related. It was actually just looping through the data and rendering it.

